Assignment 4: Banking Account and Transaction Management System

Course: PHP Web Development

Chapters: Chapter 3 (PHP Functions) and Chapter 4 (Forms, Validation and File Uploading)

Duration: 2 Hours and 30 Minutes

Total Marks: 5

Assignment Type: Individual Practical Assignment

Assignment Objective

Develop a Banking Account and Transaction Management System using HTML, CSS, and PHP. The system must allow users to enter customer information, select an account type, perform a deposit or withdrawal, calculate account balances and banking charges using PHP functions, upload an identification document, and display transaction details in an HTML table.

This is a simulation using fictional customer information and balances. It must not process real banking transactions.

Task 1: Create Banking Account Form

Create an HTML file named index.html containing the following fields:

Field	Input Type
Customer Full Name	Text
Account Number	Text
Gender	Radio Button
Account Type	Drop-down List
Opening Balance	Number
Transaction Type	Radio Button
Transaction Amount	Number
Account Duration (Years)	Number
Identification Document	File Upload

Requirements:

1. Account Type must include Savings, Current, and Business.
2. Transaction Type must include Deposit and Withdrawal.
3. Use the POST method to submit information.
4. Use enctype="multipart/form-data" for document uploading.
5. Submit the form to process.php.
6. Include Process Transaction and Reset buttons.
7. Use basic CSS to design the form.

Task 2: Create PHP Functions

Create a PHP file named functions.php containing the following eight functions.

Function	Description
calculateDeposit()	Add transaction amount to opening balance
calculateWithdrawal()	Subtract transaction amount from opening balance
calculateTransactionFee()	Calculate fee based on transaction type
calculateInterest()	Calculate interest based on account type
calculateTotalCharges()	Calculate total banking charges
calculateFinalBalance()	Calculate balance after transaction and charges
checkAccountStatus()	Determine account balance category
calculateFutureBalance()	Calculate projected balance using a default parameter

Calculation Rules

1. Deposit and Withdrawal

Use the following formulas:

- Deposit Balance = Opening Balance + Transaction Amount
- Withdrawal Balance = Opening Balance − Transaction Amount

2. Transaction Fee

Transaction Type	Transaction Fee
Deposit	No Fee
Withdrawal below $500	$2
Withdrawal of $500 or more	$5

3. Annual Interest Rate

Calculate simple interest based on account type.

Account Type	Annual Interest Rate
Savings	5%
Current	2%
Business	3%

Formula:

Interest = Final Balance × Interest Rate × Account Duration

Interest is a separate projected benefit and is not added to the current final balance.

4. Total Banking Charges

For this assignment, the transaction fee is the only charge.

Total Charges = Transaction Fee

5. Final Account Balance

- Deposit: Final Balance = Opening Balance + Deposit − Total Charges
- Withdrawal: Final Balance = Opening Balance − Withdrawal − Total Charges

6. Account Balance Status

Final Balance	Account Status
$5,000 or more	High Balance
$1,000 to less than $5,000	Medium Balance
Below $1,000	Low Balance

7. Projected Future Balance

Calculate the projected balance after simple interest:

Projected Future Balance = Final Balance + Interest

Use a default argument of 1 year in calculateFutureBalance(). The account duration entered by the customer should be passed when a different duration is required.

Function Requirements:

1. Define all eight functions in functions.php.
2. Use parameters and arguments.
3. Return results using return.
4. Use if...elseif...else statements for transaction fees, account interest rates, and account status.
5. Demonstrate a default argument.
6. Call the functions from process.php.
7. Use require_once to include functions.php.
8. Include comments explaining each function.

Task 3: Form Processing and Validation

Create process.php and retrieve submitted information using $_POST.

Apply the following validation rules:

1. Customer Full Name must not be empty.
2. Account Number must not be empty.
3. Gender must be selected.
4. Account Type must be selected.
5. Opening Balance must be zero or greater.
6. Transaction Type must be selected.
7. Transaction Amount must be greater than zero.
8. Account Duration must be a non-negative whole number.
9. Withdrawal Amount plus Transaction Fee must not exceed the available Opening Balance.
10. An identification document must be selected.
11. Display an appropriate error message when validation fails.
12. Do not process an invalid transaction or display a success message.

Use isset(), empty(), comparison operators, and conditional statements.

Task 4: Upload Customer Identification Document

Create an uploads folder.

Implement the following requirements:

1. Allow the customer to upload one identification document.
2. Retrieve the uploaded file information using $_FILES.
3. Retrieve the file name, size, type, and temporary filename.
4. Reject files larger than 2 MB.
5. Check whether a file with the same name already exists using file_exists().
6. Save the document using move_uploaded_file().
7. Do not overwrite an existing file.
8. Display success or error messages.
9. Display the file size in bytes.

Task 5: Display Banking Transaction Results (15 Marks)

After successful validation and document uploading, display the following information in an HTML table.

Banking Information	Displayed Value
Customer Full Name	Entered Name
Account Number	Entered Number
Gender	Selected Gender
Account Type	Selected Account
Opening Balance	Entered Balance
Transaction Type	Deposit or Withdrawal
Transaction Amount	Entered Amount
Transaction Fee	Calculated Fee
Total Banking Charges	Calculated Charges
Final Balance	Calculated Balance
Account Status	High, Medium or Low Balance
Account Duration	Entered Years
Interest Rate	Selected Rate
Calculated Interest	Projected Interest
Projected Future Balance	Calculated Projection
Document Name	Uploaded Filename
File Type	Uploaded File Type
File Size	Size in Bytes
Upload Status	Successful

Add a heading titled Banking Transaction Receipt and include a Back to Banking Form link.

General Instructions

1. Use HTML, CSS, and PHP only.
2. Keep the HTML form, PHP functions, and processing code in separate files.
3. Use only concepts covered in Chapters 3 and 4.

4. Do not use JavaScript, databases, or frameworks.
5. Do not create custom arrays or use array-processing loops.
6. Use $_POST and $_FILES for form processing and uploading.
7. Do not use is_numeric(), filter_input(), trim(), preg_match(), htmlspecialchars(), basename(), strpbrk(), is_dir(), or type casting.
8. Use PHP functions for banking calculations.
9. Use conditional statements for calculations and validation.
10. Include comments explaining important code sections.
11. Display results in an HTML table.
12. Test using XAMPP with fictional customer information and harmless sample documents.